



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Проф. др Игор Дејановић

Шаблон и упутство за писање завршних радова

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - мастер рад
Аутор, АУ:	Уписати име и презиме
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Шаблон и упутство за писање завршних радова
Језик публикације, ЈП:	српски/хирилица
Језик извода, ЈИ:	српски/енглески
Земља публиковања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	3/19/10/2/0/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Turpst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	
Председник:	Др Петар Петровић, ванредни професор
Члан:	Др Марко Марковић, доцент
Члан:	
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Upisati ime i prezime na latinici	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Template and tutorial for thesis preparation	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	3/19/10/2/0/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	Template, thesis, tutorial	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	01.01.2025	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	2
1.2	Предмет и циљ рада	2
1.3	Основне функционалности алата RustyJudge	3
1.4	Организација рада	3
2	Статичка анализа и алати за проверу JavaScript кода	5
2.1	Улога linter-а	5
2.2	Постојећи алати	6
2.3	Нивои информација у анализи	6
2.4	Положај RustyJudge-а	7
3	Закључак	9
	Списак табела	11
	Списак коришћених скраћеница	13
	Списак коришћених појмова	15
	Биографија	17
	Литература	19

Глава 1

Увод

Савремени софтверски системи се у великој мери ослањају на језике и алате који омогућавају брз развој, једноставну интеграцију и извршавање на различитим платформама. Један од најзначајнијих језика у том контексту је JavaScript, који се користи у развоју веб апликација, серверских система, алата командне линије, десктоп апликација и проширења за развојна окружења. Због широке примене, JavaScript код често настаје у великим и дуговечним пројектима, у којима га мења више програмера и у којима грешке у коду могу да имају значајан утицај на поузданост, одрживост и разумљивост система.

Флексибилност JavaScript језика истовремено представља и предност и изазов. Језик дозвољава различите стилове програмирања, динамичку природу вредности, слободнију употребу променљивих и велики број синтаксичких конструкција. Таква флексибилност убрзава развој, али може да отежа уочавање грешака пре извршавања програма. Неке неправилности су синтаксичке и лако се откривају већ током парсирања кода. Међутим, велики број проблема није видљив само из појединачне синтаксичке конструкције. За њихово откривање потребно је разумети односе између наредби, ток извршавања програма, области важења идентификатора и начин на који се вредности дефинишу и користе.

Један од приступа за рано откривање таквих проблема је статичка анализа програмског кода. Статичка анализа обухвата поступке у којима се програм испитује без његовог покретања. У пракси се ова идеја често примењује кроз алате познате као linter-и. Linter-и анализирају изворни код и пријављују потенцијалне грешке, сумњиве обрасце, некоришћене делове програма, непожељан стил писања или конструкције које могу да доведу до тежег одржавања. Посебно су корисни када се укључе у свакодневни ток рада, јер програмеру дају повратну информацију пре него што грешка стигне до тестирања или продукционог окружења.

У овом раду приказан је развој алата RustyJudge, linter-а за JavaScript написаног у програмском језику Rust. Алат је осмишљен као статички анализатор који, поред основне синтаксичке обраде, гради унутрашње представе програма потребне за сложенија правила провере. Посебна пажња посвећена је изградњи контролно-точног графа (CFG, енг. *Control-Flow Graph*), анализи достижности делова програма и анализи живости података (енг. *liveness analysis*). На основу тих информација мо-

гуће је имплементирати правила која не зависе само од облика једне синтаксичке конструкције, већ и од начина на који се програм може извршавати.

1.1 Мотивација

Основна мотивација за развој алата RustyJudge произилази из потребе да се поступак статичке анализе JavaScript кода прикаже као целовит процес. Уместо да се linter посматра само као листа правила, у раду се прати пут од изворног кода до дијагностике: парсирање, изградња унутрашњих структура, покретање правила и приказ резултата кориснику. Такав приказ омогућава да се јасно види које информације су потребне за одређену врсту провере.

Програмски језик Rust изабран је због карактеристика које су погодне за развој системских и развојних алата. Он омогућава високе перформансе, експлицитно управљање подацима и строжију контролу над исправношћу програма у време компајлирања. За алат који обрађује изворни код, гради графове и покреће више правила анализе, ове особине су значајне јер утичу на брзину извршавања, организацију података и поузданост имплементације.

1.2 Предмет и циљ рада

Предмет рада је пројектовање и имплементација алата RustyJudge за статичку анализу JavaScript кода. Алат чита изворне датотеке, парсира их, гради унутрашње структуре потребне за анализу, покреће скуп правила и кориснику приказује дијагностике. Дијагностика представља поруку о уоченом проблему, заједно са позицијом у изворном коду и описом правила које је прекршено.

Циљ рада је да се прикаже како се један linter може изградити као компајлерски оријентисан алат. То подразумева да се имплементација не заснива само на директном обиласку синтаксичког стабла, већ и на изградњи богатијих модела програма. Посебни циљеви рада су:

- приказ поступка парсирања JavaScript кода и добијања AST структуре;
- изградња семантичког контекста са информацијама о симболима и областима важења;
- изградња CFG представе погодне за анализу тока извршавања;
- имплементација анализе достижности и анализе живости података;
- дефинисање инфраструктуре за извршавање правила;
- имплементација конкретних правила linter-a;
- приказ дијагностика кроз CLI, LSP и VS Code проширење;
- поређење перформанси са алатима ESLint и RSLint.

Овако постављен циљ омогућава да се RustyJudge посматра из два угла. Са једне стране, он је практичан алат који може да анализира JavaScript датотеке и прикаже

кориснику проблеме у коду. Са друге стране, он је пример примене концепата из области компајлера и статичке анализе на конкретан програмски језик и конкретан развојни алат.

1.3 Основне функционалности алата RustyJudge

RustyJudge је организован као Rust пројекат који садржи библиотечки део, извршне улазе за CLI и LSP, скуп правила, модуле за CFG и анализу података, benchmark окружење и VS Code проширење. Основни ток рада почиње читањем JavaScript изворног кода и његовим парсирањем. Након тога се над добијеном структуром покрећу анализе и правила која производе дијагностике.

Унутар алата раздвојене су фазе анализе. Један део система задужен је за парсирање и обилазак синтаксичке структуре. Други део гради семантички контекст са информацијама о симболима и областима важења. Трећи део формира CFG и податке потребне за анализу живости. Над тим структурама покрећу се правила која производе дијагностике. Оваква подела омогућава да се свако правило ослони на онај ниво информација који му је стварно потребан.

Поред анализе у командној линији, RustyJudge садржи и интеграцију са развојним окружењем. LSP сервер омогућава да се резултати анализе прикажу у едитору, док VS Code проширење служи као клијентска страна која повезује корисника, отворену датотеку и RustyJudge анализатор. На тај начин се дијагностике могу приказати директно на месту где програмер пише код, што је природан начин употребе linter-a у свакодневном развоју.

Значајан део рада представља и benchmark анализа. Пошто постоје већ познати алати за анализу JavaScript кода, RustyJudge је поређен са постојећим решењима. Ово поређење не служи само да покаже време извршавања, већ и да се објасни контекст мерења: које су датотеке анализиране, која правила су покренута, како су алати конфигурисани и која су ограничења таквог поређења.

1.4 Организација рада

Рад је организован тако да се од општих појмова постепено прелази ка конкретној имплементацији алата RustyJudge. У другом поглављу приказани су појам статичке анализе, улога linter-a у развоју софтвера и постојећи алати за анализу JavaScript кода, са посебним освртом на ESLint и RSLint.

У трећем поглављу изложене су теоријске основе потребне за разумевање имплементације. Описани су AST, области важења, CFG, основни блокови, достижност, ток података и анализа живости. Ово поглавље поставља основу за касније разумевање начина на који RustyJudge представља и анализира програм.

Четврто поглавље описује архитектуру RustyJudge-а. Приказан је ток података од улазне JavaScript датотеке до дијагностике, као и улога главних модула у систему. Пето поглавље посвећено је имплементацији CFG-а и начину на који се поједине JavaScript конструкције преводе у граф тока извршавања.

У шестом поглављу описана је анализа живости података и њена примена у оквиру алата. Седмо поглавље приказује инфраструктуру за правила linter-а и конкретна правила која су имплементирана. Осмо поглавље описује употребу алата кроз CLI, LSP и VS Code проширење.

Девето поглавље посвећено је евалуацији и поређењу перформанси са алатима ESLint и RSLint. У њему су приказани услови мерења, резултати и ограничења поређења. На крају, у закључку су сумирани резултати рада, наведена ограничења тренутне имплементације и предложени правци даљег развоја.

Глава 2

Статичка анализа и алати за проверу JavaScript кода

Статичка анализа је поступак испитивања програмског кода без његовог извршавања. Циљ такве анализе је да се из самог текста програма, његове синтаксичке структуре и додатних унутрашњих представа изведу закључци о могућим грешкама, недостижном коду, непотребним деловима програма или непожељним обрацима писања. У теорији програмских језика и компајлера ова област је позната као анализа програма (енг. *program analysis*) [1].

За разлику од динамичког тестирања, статичка анализа не захтева покретање програма нити припрему улазних података. Она зато може да се извршава рано у развоју, на пример приликом чувања датотеке, пре покретања тестова или у оквиру система за континуирану интеграцију. Са друге стране, статичка анализа има ограничења. Пошто се програм не извршава, резултат анализе зависи од модела који алат гради о програму. Једноставнији модел даје бржу анализу, али мање прецизне закључке. Богатији модел омогућава сложенија правила, али повећава сложеност имплементације.

У случају JavaScript језика, статичка анализа је посебно значајна јер се језик користи у различитим окружењима и подржава велики број програмских стилова. Званична спецификација језика дефинисана је стандардом ECMA-262 [2]. Међутим, практична провера кода у развојном окружењу не своди се само на проверу да ли је програм синтаксички исправан. Потребно је открити и код који је технички исправан, али указује на могућу грешку или слабију одрживост.

2.1 Улога linter-a

Linter је практичан облик статичког анализатора. Он чита изворни код, покреће скуп правила и враћа дијагностике. Дијагностика обично садржи место проблема, кратку поруку и ознаку правила које је пријавило проблем. У зависности од алата, дијагностика може да има различит ниво озбиљности, као што су грешка, упозорење или информација.

Вредност linter-a није само у томе што проналази проблеме, већ и у тренутку у коме их пријављује. Ако се покреће у едитору, програмер добија повратну информацију док пише код. Ако се покреће у CLI окружењу, исти скуп правила може да се

користи локално, у скриптама или у систему за континуирану интеграцију. На тај начин linter постаје део развојног процеса, а не само алат који се покреће на крају рада.

Квалитет linter-a зависи од неколико особина. Дијагностике треба да буду довољно прецизне да програмер брзо пронађе узрок проблема. Број лажно позитивних пријава мора бити низак, јер се у супротном упозорења игноришу. Анализа мора бити довољно брза да не прекида рад у едитору. Правила треба да буду јасно именована и, када је могуће, подесива. Ове особине су практичне, али директно утичу на архитектуру алата: подаци морају бити организовани тако да правила имају довољно информација, али да анализа остане брза.

2.2 Постојећи алати

Најпознатији linter у JavaScript екосистему је ESLint. Он је проширив алат који се заснива на правилима, конфигурацији и могућности додавања plugin-a. Документација ESLint-a правила описује као основне градивне елементе алата: свако правило проверава да ли код испуњава одређено очекивање и пријављује проблем када то није случај [3]. Због великог броја уграђених правила и plugin-a, ESLint је у пракси чест избор за JavaScript и TypeScript пројекте.

ESLint је други значајан алат у контексту овог рада. У питању је JavaScript linter написан у Rust-у, са нагласком на брзину, мању потрошњу меморије и практичан CLI рад [4]. За RustyJudge је ESLint посебно релевантан зато што омогућава поређење са алатом из сличног технолошког простора. У benchmark делу рада ESLint се користи кроз CLI, па се мери укупно време извршавања, без раздвајања унутрашњих фаза као што су парсирање и извршавање правила.

Табела 1: Поређење алата релевантних за рад.

Алат	Разлог за поређење
ESLint	Представља широко коришћено решење из праксе и добар оријентир за понашање JavaScript linter-a.
ESLint	Омогућава поређење са алатом који је технолошки ближи RustyJudge-у.
RustyJudge	Предмет је имплементације и омогућава мерење појединачних фаза анализе.

Табела 1 приказује улогу сваког алата у овом раду.

2.3 Нивои информација у анализи

Правила linter-a могу да се посматрају кроз ниво информација који им је потребан. Не захтева свако правило исту дубину анализе. Нека правила могу да се изврше над

појединачним чвором синтаксичког стабла, док друга морају да користе области важења, ток извршавања или ток података. Ова разлика је важна јер утиче на цену анализе и на организацију имплементације.

Табела 2: Нивои информација потребни за различите врсте lint правила.

Ниво анализе	Информације	Пример провере
Синтаксички	Облик израза, наредби и декларација.	Препознавање непожељне синтаксичке конструкције.
Семантички	Веза између употребе имена и његове декларације.	Откривање некоришћене декларације или погрешне употребе идентификатора.
Контролно-проточни	Могуће путање извршавања програма.	Откривање недостижног кода после наредбе која прекида ток.
Ток података	Места на којима се вредност дефинише, користи или престаје да буде потребна.	Откривање доделе чији резултат касније није употребљен.

Табела 2 показује да избор унутрашње представе није само технички детаљ, већ одређује које врсте правила алат може природно да подржи. Зато је корисно да linter има јасно раздвојене слојеве анализе. Једноставна правила не треба да плаћају цену сложених анализа, док сложенија правила морају имати приступ подацима који превазилазе непосредни облик кода.

2.4 Положај RustyJudge-a

RustyJudge није замишљен као потпуна замена за ESLint. ESLint има велики број правила, развијен plugin екосистем и дугу примену у реалним пројектима. RustyJudge има ужи циљ: да прикаже како се linter може изградити око јасно одвојених фаза анализе и како се те фазе могу искористити за правила која зависе од тока извршавања и тока података.

У односу на постојеће алате, вредност RustyJudge-a у овом раду није у броју подржаних правила, већ у транспарентности архитектуре. Фазе анализе су раздвојене тако да се може мерити време парсирања, изградње семантичког модела и CFG-a, извршавања правила и укупног linting процеса. То је важно за benchmark поглавље, јер омогућава да се резултати не посматрају само као један број, већ као збир више корака.

Овакво позиционирање одређује и остатак рада. Наредно поглавље зато не улази одмах у имплементацију, већ уводи појмове који су потребни да би се она разумела: AST, области важења, CFG, достижност, ток података и анализа живости. Након тога се приказује како су ти појмови реализовани у *RustyJudge-u*.

Глава 3

Закључак

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак табела

Табела 1	Поређење алата релевантних за рад.	6
Табела 2	Нивои информација потребни за различите врсте lint правила.	7

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] F. Nielson, H. R. Nielson, and C. Hankin, *Principles of Program Analysis*. Springer, 1999.
- [2] Ecma International, “ECMA-262 ECMAScript Language Specification.” Accessed: June 28, 2026. [Online]. Доступно на <https://tc39.es/ecma262/>
- [3] OpenJS Foundation and ESLint contributors, “Core Concepts - ESLint.” Accessed: June 28, 2026. [Online]. Доступно на <https://eslint.org/docs/latest/use/core-concepts/>
- [4] RSLint contributors, “RSLint - GitHub repository.” Accessed: June 28, 2026. [Online]. Доступно на <https://github.com/rslint/rslint>